

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



DEPARTMENT OF COMPUTER SCIENCE

REPORT OF PROJECT 1

Project: Data Fitting and Ordinary Least Squares Method

Lecturer:	ThS. Võ Nam Thực Đoàn	
	ThS. Lê Nhật Nam	
Leader:	Khang Phúc Nguyen	24120068
Members:	Nghia Trong Hoang	24120103
	Nhat Hoang Mai	24120109
	Nhat Hoang Nguyen	24120110
	Long Nhat Phung Vo	24120088

Ho Chi Minh City, 4/2026

Table of Contents

1	Bài Toán Data Fitting và Phương Pháp OLS	1
1.1	Phát Biểu Bài Toán Data Fitting Tổng Quát	1
1.1.1	Đặt vấn đề	1
1.1.2	Mô hình hồi quy tuyến tính	1
1.2	Các Giả Thiết Gauss–Markov	2
1.3	Hàm Mất Mát RSS và Chứng Minh Nghiệm OLS	3
1.3.1	Hàm mất mát RSS	3
1.3.2	Chứng minh nghiệm OLS (Normal Equations)	4
1.4	Ma Trận Chiều (Hat Matrix)	7
1.4.1	Định nghĩa và hình học	7
1.4.2	Các tính chất của Hat Matrix	8
1.5	Ước Lượng Phương Sai Nhiều σ^2	9
1.6	Định Lý Gauss–Markov (BLUE)	10
1.7	Đánh Giá Mô Hình	11
1.7.1	Phân rã phương sai và hệ số xác định	11
1.7.2	Kiểm định giả thuyết	12
2	Cài Đặt Python	13
2.1	Imports, hằng số, dataclass và các hàm phụ trợ	13
2.2	Hàm <code>ols_fit(X, y)</code>	17
2.2.1	Mô tả	17
2.2.2	Code	17
2.3	Hàm <code>hat_matrix(X)</code>	19
2.3.1	Mô tả	19
2.3.2	Code	20
2.4	Kiểm thử và kiểm chứng	22

1 Bài Toán Data Fitting và Phương Pháp OLS

1.1 Phát Biểu Bài Toán Data Fitting Tổng Quát

Bài toán Data Fitting đã có nền tảng toán học và lịch sử từ lâu đời, nó luôn đi chung với sự phát triển của phương pháp **Bình Phương Tối Thiểu (OLS)**. Bài toán được hình thành được đặt ra không chỉ để thỏa mãn niềm đam mê toán học của các nhà nghiên cứu mà nó còn có mục đích rất thực tiễn trong các ngành liên quan về thiên văn học và trắc địa vào cuối thế kỷ 18 và đầu thế kỷ 19.

1.1.1 Đặt vấn đề

Vào những năm 1790, để có thể thiết lập ra hệ mét¹ thì ông đã phải đo đạc chính xác chiều dài **cung kinh tuyến** từ cực Bắc đến Xích đạo đi qua Paris [1]. Họ đã không trực tiếp mà họ đo một đoạn cung kinh tuyến, cụ thể là đoạn từ *Runkirk* đến *Barcelona*, rồi dùng mô hình toán học để suy ra độ dài **một phần tư kinh tuyến Trái Đất**.

Do đó ta có thể thấy rằng, bài toán **Data Fitting** đã xuất hiện từ rất lâu và nó đã trở thành một trong những bài toán kinh điển trong Toán học và là nền tảng cho các mô hình học máy sau này. Cụ thể hơn, ta có một tập dữ liệu quan sát được và muốn tìm một hàm số mô tả mối quan hệ giữa đầu vào và đầu ra. Bài toán này được gọi chung là **data fitting** (khớp dữ liệu).

Định nghĩa 1.1: Bài toán Data Fitting tổng quát

Cho tập dữ liệu $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ với $\mathbf{x}_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}$. Bài toán data fitting là tìm hàm $f: \mathbb{R}^p \rightarrow \mathbb{R}$ trong một lớp hàm cho trước $\mathcal{F}$ sao cho f xấp xỉ tốt nhất ánh xạ từ $\mathbf{x}_i$ đến y_i theo một tiêu chí tối ưu:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \mathcal{L}(f; \mathcal{D}),$$

trong đó $\mathcal{L}$ là hàm mất mát (*loss function*) đo mức độ sai lệch giữa dự đoán của mô hình và giá trị thực.

1.1.2 Mô hình hồi quy tuyến tính

Mô hình hồi quy tuyến tính là nền tảng cốt lõi trong toán học thống kê, nó có lịch sử phát triển gắn với bài toán Data Fitting và phương pháp bình phương tối thiểu. Mục tiêu của mô hình là có thể *ước lượng các tham số chưa biết thông qua các dữ liệu thực nghiệm đã quan sát*

¹ Hệ mét là hệ thống đo lường thập phân được thống nhất rộng rãi trên quốc tế. Dù có nhiều biến thể của hệ thống số liệu nổi lên vào cuối thế kỷ XIX và đầu thế kỷ XX, thuật ngữ này thường được dùng như một từ đồng nghĩa là "SI" hoặc "Hệ thống đơn vị Quốc Tế" Applied Mathematics in Statistics

được [2]. Trong mô hình này, các quan sát được biểu diễn dưới dạng một **tổ hợp tuyến tính** của các biến đã biết và các **tham số ẩn**.

Giả sử ta có một chuỗi các quan sát y_i phụ thuộc vào một mô hình tuyến tính $\alpha + \beta x_i$ ($i = 1, \dots, n$). Bài toán đặt ra là làm sao ta có thể ước lượng các giá trị $\hat{\alpha}$ và $\hat{\beta}$ cho các tham số α và β để có thể phù hợp với bộ dữ liệu quan sát được một cách tốt nhất và ít chi phí nhất [2]. Tổng quát bài toán, thì phần lớn ta sẽ có thể viết mô hình dưới dạng $Y = M\beta + \theta$ và cụ thể hơn $\mathcal{F}$ là tập các hàm tuyến tính theo tham số [3]. Khi đó, giả thiết:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^T \boldsymbol{\beta} + \varepsilon_i,$$

với $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^T \in \mathbb{R}^{p+1}$ là vector tham số cần ước lượng và ε_i là nhiễu ngẫu nhiên (sai số mô hình).

Đặt ma trận thiết kế (*design matrix*) $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$ trong đó hàng thứ i là $(1, x_{i1}, \dots, x_{ip})$ (cột đầu tiên toàn số 1 ứng với hệ số tự do β_0). Khi đó toàn bộ hệ có thể viết gọn:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \quad \mathbf{y} \in \mathbb{R}^n, \mathbf{X} \in \mathbb{R}^{n \times (p+1)}, \boldsymbol{\varepsilon} \in \mathbb{R}^n. \quad (1)$$

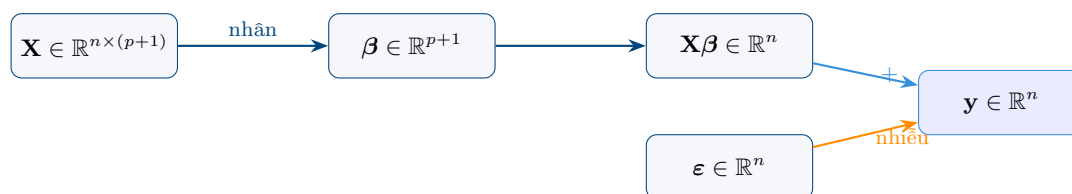


Figure 1: Cấu trúc mô hình hồi quy tuyến tính dạng ma trận: $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$. Dữ liệu quan sát $\mathbf{y}$ được phân tích thành thành phần *tín hiệu* $\mathbf{X}\boldsymbol{\beta}$ (xanh) và thành phần *nhiều* $\boldsymbol{\varepsilon}$ (cam).

1.2 Các Giả Thiết Gauss–Markov

Trong quá trình phát triển của phương pháp OLS, ban đầu Gauss đã đề xuất ra một cách chứng minh dựa trên nền tảng xác suất và phân phối. Nhưng mà cách chứng minh này lại có hạn chế là ép đặt phân phối sai số ban đầu phải tuân theo phân phối chuẩn hình chuông. Sau đó chính bản thân ông đã nhận ra sai lầm của mình mà từ đó đề xuất ra một cách chứng minh khác đó là Gauss-Markov dựa trên các giả thiết sau:

Để ước lượng OLS có các tính chất thống kê tốt, mô hình tuyến tính (1) cần thỏa mãn năm giả thiết Gauss–Markov sau [4, 5]. Các giả thiết này xác định khuôn khổ mà trong đó OLS được đảm bảo là ước lượng tốt nhất.

Định nghĩa 1.2: Các giả thiết Gauss–Markov (GM1–GM5)

1. **Tuyến tính:** $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$. Mô hình tuyến tính theo tham số $\boldsymbol{\beta}$; cho phép áp dụng đại số tuyến tính để giải bài toán tối ưu.
2. **Không hoàn hảo đa cộng tuyến:** $\text{rank}(\mathbf{X}) = p + 1$, tức các cột của $\mathbf{X}$ độc lập tuyến tính và $\mathbf{X}^T \mathbf{X}$ khả nghịch. Nếu vi phạm, nghiệm OLS không xác định duy nhất.
3. **Ngoại sinh (zero conditional mean):** $\mathbb{E}[\boldsymbol{\varepsilon} | \mathbf{X}] = \mathbf{0}$, tức $\mathbb{E}[\varepsilon_i | \mathbf{x}_i] = 0$ với mọi i . Đảm bảo không có biến quan trọng bị bỏ sót có tương quan với $\mathbf{X}$.
4. **Đồng phương sai (homoscedasticity):** $\text{Var}(\boldsymbol{\varepsilon} | \mathbf{X}) = \sigma^2 \mathbf{I}_n$, tức $\text{Var}(\varepsilon_i) = \sigma^2$ và $\text{Cov}(\varepsilon_i, \varepsilon_j) = 0$ với $i \neq j$. Nếu vi phạm (heteroscedasticity), sai số chuẩn bị lệch.
5. **Phần dư chuẩn (normality):** $\boldsymbol{\varepsilon} | \mathbf{X} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n)$. Điều kiện bổ sung — cần cho kiểm định t và F trong mẫu nhỏ; với mẫu lớn có thể nới lỏng nhờ định lý giới hạn trung tâm.

Giả thiết 1–4 đủ để đảm bảo OLS là **BLUE** (Best Linear Unbiased Estimator) theo Định lý Gauss–Markov (xem mục 1.6 bên dưới). Giả thiết 5 chỉ cần thiết khi thực hiện kiểm định giả thuyết và xây dựng khoảng tin cậy.

1.3 Hàm Mất Mát RSS và Chứng Minh Nghiệm OLS

Hàm mất mát Residual Sum of Squares được gọi là tổng bình phương các sai số, là mục tiêu cốt lõi khi các nhà toán học làm việc với bài toán Data Fitting, họ sẽ cố gắng **tối thiểu hóa** hàm mất mát này để có thể tạo ra mô hình tốt nhất cho tập dữ liệu mà họ có, điều này rất phổ biến trong phương pháp bình phương tối thiểu và hồi quy.

1.3.1 Hàm mất mát RSS

Vào khoảng năm 1805, nhà toán học Andrien-Marie Legendre đã xây dựng một hệ phương trình tuyến tính cho các quan sát mà ông có được, trong đó mỗi phương trình của hệ có một sai số E . Hàm mất RSS được ông ký hiệu là S đại diện cho tổng bình phương của tất cả các sai số

này với $S = E^2 + E'^2 + E''^2 + \dots$ [2]

Định nghĩa 1.3: Residual Sum of Squares (RSS)

Hàm mất mát của phương pháp Ordinary Least Squares (OLS) là tổng bình phương phần dư (*Residual Sum of Squares*):

$$\text{RSS}(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2 = \sum_{i=1}^n (y_i - \mathbf{x}_i^T \beta)^2.$$

OLS tìm $\hat{\beta}$ tối thiểu hóa $\text{RSS}(\beta)$.

RSS đo tổng bình phương khoảng cách từ các điểm quan sát y_i đến các giá trị dự đoán $\hat{y}_i = \mathbf{x}_i^T \beta$ của mô hình. OLS tìm hyperplane tốt nhất theo nghĩa cực tiểu hóa tổng khoảng cách vuông góc theo chiều y .

Với sự ra đời của RSS thì bài toán OLS đã trở nên rõ ràng hơn, nó đã cung cấp một công cụ mạnh mẽ để ước lượng các tham số của mô hình hồi quy tuyến tính, từ đó giúp các nhà nghiên cứu có thể hiểu rõ hơn về mối quan hệ giữa các biến và đưa ra dự đoán chính xác hơn dựa trên dữ liệu quan sát được. (1) *Tạo ra sự cân bằng và triệt tiêu sai số cực đoan.* Điều này đã được Legendre lập luận và chứng minh rằng hàm bình phương mang lại một sự cân bằng lớn giữa các sai số. Nó đưa ra điểm phạt rất lớn cho các sai số lớn từ đó mà ngăn được các phần tử ngoại lai giúp ta tìm ra trạng thái gần thực tế nhất. (2) *Dễ dàng tính được bằng giải tích.* Khác với hàm giá trị tuyệt đối khác gặp khó khăn lớn trong việc tính toán thì RSS lại mang lại khả năng tính toán vô cùng thuận lợi. (3) *Nên tăng xác suất vững chắc.* RSS đã được Gauss chứng minh hoàn hảo thông qua thống kê, ông đã chứng minh rằng nếu các phần dư tuân theo quy luật phân phối chuẩn thì việc cực tiểu hóa hàm mất mát RSS hoàn toàn tương đương với **maximum likelihood** [2].

1.3.2 Chứng minh nghiệm OLS (Normal Equations)

Xuyên suốt lịch sử phát triển, quá trình chứng minh phương pháp Bình Phương tối thiểu là một cuộc hành trình dài từ đầu thế kỷ 19, nó đã đi từ những lập luận đại số thuần túy cho tới nền tảng xác suất và cuối cùng chỉ thật sự được chứng minh tối ưu thông qua hình học trực giao

và đại số tuyến tính ma trận hiện đại.

Vào những năm đầu tiên thuở sơ khai của phương pháp, nhà toán học người pháp Legendre đã đề xuất phương pháp OLS kèm theo một cách chứng minh bằng đại số thuần túy, không có yếu tố xác suất đi cùng. Để tìm được giá trị nhỏ nhất của S , ông lấy đạo hàm riêng của S theo từng ẩn số $(x, y, z, \dots)$ và cho chúng bằng 0. Nhờ đó quá trình này đã tạo ra một hệ phương trình tuyến tính mới có nghiệm duy nhất. Tuy nhiên, cách chứng minh này vẫn còn tồn đọng hạn chế là ông chỉ biện minh phương pháp này dựa trên lý luận thực nghiệm rằng bình phương sẽ phạt các sai số lớn, tạo ra 'sự cân bằng' giữa các quan sát [2] mà lại không đưa ra được độ chính xác hay sai số của các hệ số tìm được.

Sau đó vào năm 1809, Carl Friedrich Gauss đã đưa ra cách chứng minh của mình được xuất bản bên trong cuốn *Theoria Motus Corporum Coelestium*. Ông đã đưa OLS lên một nền tảng xác suất vững chắc hơn. Gauss sử dụng nguyên lý **Hợp lý cực đại** để chứng minh bài toán này. Ông giả sử các sai số đo đạc là các biến ngẫu nhiên độc lập, và từ đó mà đã tìm ra một quy luật phân phối sai số sao cho mà giá trị trung bình cộng chính là giá trị có xác suất xảy ra cao nhất. Điều này đã dẫn tới việc ông tìm ra hàm **Mật độ xác suất phân phối chuẩn**. Khi mà các sai số tuân theo phân phối chuẩn, việc cực đại hóa hàm hợp lý hoàn toàn tương đương về mặt toán học với việc **cực tiểu hóa tổng bình phương các sai số**.

Và sau Gauss đã có rất nhiều người tham gia vào chứng minh và sửa các hạn chế mà người đi trước mắc phải. Chẳng hạn như sau đó ông Laplace đã đề xuất cách chứng minh để khắc phục hạn chế của Gauss khi ông đang ép buộc sai số phải tuân theo phân phối chuẩn. Laplace đã áp dụng một phiên bản sơ khai của **Định lý giới hạn trung gian** chứng minh rằng khi số lượng quan sát là rất lớn, thì sự phân phối của tổng hoặc trung bình các sai số sẽ luôn hội tụ về phân phối chuẩn, bất kể phân phối gốc của từng sai số đơn lẻ là gì. Và từ đó mà ông đã chứng minh rằng OLS của Legendre và Gauss là phương pháp mang lại độ chính xác cao nhất cho các tập dữ liệu lớn. Sau này Gauss đã đề xuất cách chứng minh khi không biết quy luật phân phối của sai số ban đầu.

Cuối cùng theo thời gian thì hiện nay ta chứng minh OLS bằng hình học trực giao và đại số ma trận, đây chính là cách được xem là tối ưu, ngắn gọn và đẹp đẽ nhất. Khi nó kết hợp **Đại số tuyến tính và Phép chiếu trực giao**, là cách tiếp cận bao trùm mọi vấn đề của OLS mà không dùng đến đạo hàm phức tạp.

Định lý 1.1: Nghiệm OLS — Normal Equations

Nếu $\mathbf{X}^T \mathbf{X}$ khả nghịch (tương đương $\text{rank}(\mathbf{X}) = p + 1$), bài toán cực tiểu hóa $\text{RSS}(\boldsymbol{\beta})$ có nghiệm duy nhất:

$$\hat{\boldsymbol{\beta}}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.$$

Chứng minh. Khai triển RSS dưới dạng tích vô hướng để áp dụng giải tích ma trận [6]:

$$\begin{aligned} \text{RSS}(\boldsymbol{\beta}) &= (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \\ &= \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\boldsymbol{\beta} - \boldsymbol{\beta}^T \mathbf{X}^T \mathbf{y} + \boldsymbol{\beta}^T \mathbf{X}^T \mathbf{X} \boldsymbol{\beta}. \end{aligned}$$

Vì $\mathbf{y}^T \mathbf{X}\boldsymbol{\beta}$ là một số thực (ma trận 1×1), nó bằng chuyển vị của chính nó: $\mathbf{y}^T \mathbf{X}\boldsymbol{\beta} = (\mathbf{y}^T \mathbf{X}\boldsymbol{\beta})^T = \boldsymbol{\beta}^T \mathbf{X}^T \mathbf{y}$. Do đó:

$$\text{RSS}(\boldsymbol{\beta}) = \mathbf{y}^T \mathbf{y} - 2\boldsymbol{\beta}^T \mathbf{X}^T \mathbf{y} + \boldsymbol{\beta}^T \mathbf{X}^T \mathbf{X} \boldsymbol{\beta}.$$

Bước 1: Lấy gradient theo $\boldsymbol{\beta}$.

Áp dụng các công thức đạo hàm ma trận chuẩn: $\nabla_{\boldsymbol{\beta}}(\mathbf{a}^T \boldsymbol{\beta}) = \mathbf{a}$ và $\nabla_{\boldsymbol{\beta}}(\boldsymbol{\beta}^T \mathbf{M} \boldsymbol{\beta}) = 2\mathbf{M}\boldsymbol{\beta}$ (với $\mathbf{M}$ đối xứng — ở đây $\mathbf{M} = \mathbf{X}^T \mathbf{X}$ đối xứng):

$$\nabla_{\boldsymbol{\beta}} \text{RSS} = \mathbf{0} - 2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \boldsymbol{\beta} = -2\mathbf{X}^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}).$$

Bước 2: Điều kiện cần tại cực trị.

Cho gradient bằng không:

$$\nabla_{\boldsymbol{\beta}} \text{RSS} = \mathbf{0} \implies \mathbf{X}^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = \mathbf{0} \iff \mathbf{X}^T \mathbf{X} \boldsymbol{\beta} = \mathbf{X}^T \mathbf{y}.$$

Đây là hệ phương trình **Normal Equations**.

Bước 3: Nghiệm duy nhất.

Vì $\mathbf{X}^T \mathbf{X}$ khả nghịch theo giả thiết:

$$\hat{\boldsymbol{\beta}}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.$$

Bước 4: Kiểm tra đây là cực tiểu (không phải cực đại hay điểm yên ngựa).

Ma trận Hessian của RSS là $\mathbf{H}_{\text{RSS}} = 2\mathbf{X}^T\mathbf{X}$. Vì $\text{rank}(\mathbf{X}) = p + 1$, với mọi $\mathbf{v} \neq \mathbf{0}$:

$$\mathbf{v}^T(\mathbf{X}^T\mathbf{X})\mathbf{v} = (\mathbf{X}\mathbf{v})^T(\mathbf{X}\mathbf{v}) = \|\mathbf{X}\mathbf{v}\|_2^2 > 0,$$

nên Hessian xác định dương. Vậy điểm tối hạn tìm được là **cực tiểu toàn cục** (và là duy nhất do hàm RSS lồi chặt). $\square$

Hệ quả 1.1: Tính chất của phần dư OLS

Phần dư OLS $\hat{\mathbf{e}} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}$ trực giao với không gian cột của $\mathbf{X}$:

$$\mathbf{X}^T\hat{\mathbf{e}} = \mathbf{0}.$$

Chứng minh. Từ Normal Equations: $\mathbf{X}^T\mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{X}^T\mathbf{y}$, suy ra $\mathbf{X}^T(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = \mathbf{0}$, tức là $\mathbf{X}^T\hat{\mathbf{e}} = \mathbf{0}$. $\square$

1.4 Ma Trận Chiều (Hat Matrix)

1.4.1 Định nghĩa và hình học

Định nghĩa 1.4: Hat Matrix (Ma trận chiều)

Ma trận chiều (hay hat matrix) là:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T \in \mathbb{R}^{n \times n}.$$

Tên gọi “hat” xuất phát từ việc $\mathbf{H}$ biến $\mathbf{y}$ thành $\hat{\mathbf{y}}$ (“đội mũ” lên y): $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{H}\mathbf{y}$.

Về mặt hình học, $\mathbf{H}$ thực hiện phép chiếu trực giao của $\mathbf{y}$ lên không gian cột của $\mathbf{X}$, ký hiệu $\mathcal{C}(\mathbf{X})$ [7]. Ma trận bù $\mathbf{I} - \mathbf{H}$ chiếu $\mathbf{y}$ lên không gian trực giao $\mathcal{C}(\mathbf{X})^\perp$, cho ra phần dư $\hat{\mathbf{e}}$.

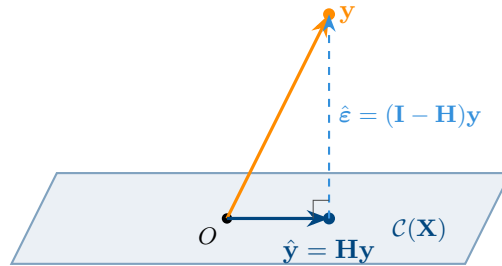


Figure 2: Hình học của Hat Matrix: $\mathbf{H}$ chiếu trực giao $\mathbf{y}$ lên không gian cột $\mathcal{C}(\mathbf{X})$, cho ra giá trị khớp $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$. Phần dư $\hat{\mathbf{e}} = (\mathbf{I} - \mathbf{H})\mathbf{y}$ vuông góc với $\mathcal{C}(\mathbf{X})$.

1.4.2 Các tính chất của Hat Matrix

Mệnh đề 1.1: Tính chất của H

[8]

- (i) **Idempotent:** $\mathbf{H}^2 = \mathbf{H}$.
- (ii) **Đối xứng:** $\mathbf{H}^T = \mathbf{H}$.
- (iii) **Giá trị riêng:** Chỉ nhận giá trị 0 hoặc 1.
- (iv) **Hạng:** $\text{rank}(\mathbf{H}) = p + 1$.
- (v) **Giá trị khớp và phần dư:** $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$; $\hat{\mathbf{e}} = (\mathbf{I} - \mathbf{H})\mathbf{y}$.

Chứng minh. (i) **Idempotent:**

$$\begin{aligned}\mathbf{H}^2 &= [\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T][\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T] \\ &= \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1} \underbrace{(\mathbf{X}^T\mathbf{X})(\mathbf{X}^T\mathbf{X})^{-1}}_{=\mathbf{I}_{p+1}} \mathbf{X}^T \\ &= \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T = \mathbf{H}.\end{aligned}$$

(ii) **Đối xứng:**

$$\mathbf{H}^T = [\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T]^T = \mathbf{X}[(\mathbf{X}^T\mathbf{X})^{-1}]^T\mathbf{X}^T.$$

Vì $\mathbf{X}^T\mathbf{X}$ đối xứng, nghịch đảo của nó cũng đối xứng: $[(\mathbf{X}^T\mathbf{X})^{-1}]^T = (\mathbf{X}^T\mathbf{X})^{-1}$. Do đó $\mathbf{H}^T = \mathbf{H}$.

(iii) **Giá trị riêng:** Với mọi giá trị riêng λ và vector riêng $\mathbf{v}$: $\mathbf{H}\mathbf{v} = \lambda\mathbf{v}$. Nhân hai vế bởi $\mathbf{H}$: $\mathbf{H}^2\mathbf{v} = \lambda\mathbf{H}\mathbf{v}$, tức $\mathbf{H}\mathbf{v} = \lambda^2\mathbf{v}$. Suy ra $\lambda\mathbf{v} = \lambda^2\mathbf{v}$, nên $\lambda(\lambda - 1) = 0$, tức $\lambda \in \{0, 1\}$.

(iv) **Hạng:** $\text{rank}(\mathbf{H}) = \text{tr}(\mathbf{H})$ (vì giá trị riêng chỉ là 0 hoặc 1). Ta có: $\text{tr}(\mathbf{H}) = \text{tr}(\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T) =$

$$\text{tr}((\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X}) = \text{tr}(\mathbf{I}_{p+1}) = p + 1.$$

(v) **Giá trị khớp và phần dư:** Theo định nghĩa, $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \mathbf{H}\mathbf{y}$. Phần dư: $\hat{\boldsymbol{\varepsilon}} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I} - \mathbf{H})\mathbf{y}$. $\square$

1.5 Ước Lượng Phương Sai Nhiễu σ^2

Định lý 1.2: Ước lượng không chệch của σ^2

Dưới giả thiết Gauss–Markov (đặc biệt $\mathbb{E}[\boldsymbol{\varepsilon}|\mathbf{X}] = \mathbf{0}$ và $\text{Var}(\boldsymbol{\varepsilon}|\mathbf{X}) = \sigma^2 \mathbf{I}_n$), ước lượng:

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n - p - 1} = \frac{\|\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}\|_2^2}{n - p - 1}$$

là ước lượng không chệch của σ^2 , tức $\mathbb{E}[\hat{\sigma}^2|\mathbf{X}] = \sigma^2$.

Chứng minh. Viết $\text{RSS} = \hat{\boldsymbol{\varepsilon}}^T \hat{\boldsymbol{\varepsilon}}$ với $\hat{\boldsymbol{\varepsilon}} = (\mathbf{I} - \mathbf{H})\mathbf{y} = (\mathbf{I} - \mathbf{H})\boldsymbol{\varepsilon}$ (vì $(\mathbf{I} - \mathbf{H})\mathbf{X}\boldsymbol{\beta} = \mathbf{0}$).

Lấy kỳ vọng:

$$\begin{aligned}\mathbb{E}[\text{RSS}|\mathbf{X}] &= \mathbb{E}[\boldsymbol{\varepsilon}^T (\mathbf{I} - \mathbf{H}) \boldsymbol{\varepsilon} | \mathbf{X}] = \text{tr}((\mathbf{I} - \mathbf{H}) \text{Var}(\boldsymbol{\varepsilon}|\mathbf{X})) \\ &= \text{tr}((\mathbf{I} - \mathbf{H}) \sigma^2 \mathbf{I}_n) = \sigma^2 \text{tr}(\mathbf{I} - \mathbf{H}) \\ &= \sigma^2 (n - \text{tr}(\mathbf{H})) = \sigma^2 (n - p - 1).\end{aligned}$$

Bước thứ nhất dùng công thức $\mathbb{E}[\mathbf{v}^T \mathbf{A} \mathbf{v}] = \text{tr}(\mathbf{A} \boldsymbol{\Sigma})$ với $\boldsymbol{\Sigma} = \text{Var}(\mathbf{v})$. Suy ra $\mathbb{E}[\hat{\sigma}^2|\mathbf{X}] = \sigma^2$. $\square$

Mẫu số $n - p - 1$ là số bậc tự do còn lại sau khi ước lượng $p + 1$ tham số từ n quan sát. Nếu dùng n ở mẫu, ước lượng sẽ bị chệch về phía nhỏ hơn (biased downward).

1.6 Định lý Gauss–Markov (BLUE)

Định lý 1.3: Gauss–Markov (BLUE)

[4] Dưới các giả thiết **GM1–GM4**, ước lượng OLS $\hat{\beta}_{OLS}$ là ước lượng tuyến tính không chệch tốt nhất (*Best Linear Unbiased Estimator* — **BLUE**):

- (i) **Không chệch:** $\mathbb{E}[\hat{\beta}_{OLS} | \mathbf{X}] = \beta$.
- (ii) **Ma trận phương sai:** $\text{Var}(\hat{\beta}_{OLS} | \mathbf{X}) = \sigma^2(\mathbf{X}^T \mathbf{X})^{-1}$.
- (iii) **Tốt nhất (phương sai nhỏ nhất):** Với mọi ước lượng tuyến tính không chệch $\tilde{\beta}$ khác, $\text{Var}(\tilde{\beta}_j) \geq \text{Var}(\hat{\beta}_j^{OLS})$ với mọi j .

Chứng minh. (i) **Tính không chệch.**

Từ Normal Equations: $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. Thay $\mathbf{y} = \mathbf{X}\beta + \varepsilon$:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T (\mathbf{X}\beta + \varepsilon) = \beta + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \varepsilon.$$

Lấy kỳ vọng có điều kiện và áp dụng GM3:

$$\mathbb{E}[\hat{\beta} | \mathbf{X}] = \beta + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \underbrace{\mathbb{E}[\varepsilon | \mathbf{X}]}_{= \mathbf{0} \text{ (GM3)}} = \beta.$$

(ii) **Ma trận phương sai.**

Đặt $\mathbf{C} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$, khi đó $\hat{\beta} - \beta = \mathbf{C}\varepsilon$. Áp dụng GM4:

$$\text{Var}(\hat{\beta} | \mathbf{X}) = \mathbf{C} \underbrace{\text{Var}(\varepsilon | \mathbf{X})}_{= \sigma^2 \mathbf{I}_n \text{ (GM4)}} \mathbf{C}^T = \sigma^2 \mathbf{C} \mathbf{C}^T = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}.$$

(iii) **Tính tốt nhất.**

Với mọi ước lượng tuyến tính không chệch $\tilde{\beta} = \mathbf{A}\mathbf{y}$ (với $\mathbf{A}$ là ma trận hằng số), đặt $\mathbf{D} = \mathbf{A} - \mathbf{C}$.

Tính không chệch đòi hỏi $\mathbf{DX} = \mathbf{0}$, suy ra $\mathbf{CD}^T = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{D}^T = \mathbf{0}^T = \mathbf{0}$. Do đó:

$$\begin{aligned}\text{Var}(\tilde{\beta} | \mathbf{X}) &= \sigma^2 \mathbf{A} \mathbf{A}^T = \sigma^2 (\mathbf{C} + \mathbf{D})(\mathbf{C} + \mathbf{D})^T \\ &= \sigma^2 (\mathbf{C} \mathbf{C}^T + \mathbf{C} \mathbf{D}^T + \mathbf{D} \mathbf{C}^T + \mathbf{D} \mathbf{D}^T) \\ &= \underbrace{\sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}}_{\text{Var}(\hat{\beta})} + \underbrace{\sigma^2 \mathbf{D} \mathbf{D}^T}_{\succeq \mathbf{0}}.\end{aligned}$$

Vì $\mathbf{D} \mathbf{D}^T$ nửa xác định dương, $\text{Var}(\tilde{\beta}) - \text{Var}(\hat{\beta}) \succeq \mathbf{0}$, tức $\text{Var}(\tilde{\beta}_j) \geq \text{Var}(\hat{\beta}_j^{\text{OLS}})$ với mọi j . $\square$

1.7 Đánh Giá Mô Hình

1.7.1 Phân rã phương sai và hệ số xác định

Việc đánh giá mô hình bắt đầu từ việc phân tích nguồn gốc biến động của biến mục tiêu $\mathbf{y}$.

Định nghĩa 1.5: Phân rã tổng bình phương

Khi mô hình có hệ số chặn β_0 , tổng bình phương toàn phần (TSS) phân rã thành:

$$\underbrace{\sum_{i=1}^n (y_i - \bar{y})^2}_{\text{TSS}} = \underbrace{\sum_{i=1}^n (\hat{y}_i - \bar{y})^2}_{\text{ESS}} + \underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{RSS}},$$

trong đó $\bar{y} = \frac{1}{n} \sum_i y_i$. **TSS** là biến động toàn phần chưa có mô hình; **ESS** là phần mô hình giải thích được; **RSS** là phần chưa giải thích.

Định nghĩa 1.6: Hệ số xác định R^2 và $\bar{R}^2$

[9, 4]

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = \frac{\text{ESS}}{\text{TSS}} \in [0, 1].$$

Vì R^2 luôn tăng (hoặc giữ nguyên) khi thêm bất kỳ biến nào, ta dùng $\bar{R}^2$ **hiệu chỉnh** để phạt việc thêm biến không có ý nghĩa:

$$\bar{R}^2 = 1 - \frac{\text{RSS}/(n-p-1)}{\text{TSS}/(n-1)} = 1 - \frac{n-1}{n-p-1}(1-R^2).$$

$\bar{R}^2$ chỉ tăng khi biến mới cải thiện mô hình đủ bù đắp bậc tự do bị mất; $\bar{R}^2$ có thể âm khi mô hình kém hơn mô hình rỗng.

1.7.2 Kiểm định giả thuyết

Dưới giả thiết GM5, $\hat{\beta} \sim \mathcal{N}(\beta, \sigma^2(\mathbf{X}^T \mathbf{X})^{-1})$.

Định lý 1.4: Kiểm định t cho từng hệ số

[4, 9] Để kiểm định $H_0 : \beta_j = 0$ so với $H_1 : \beta_j \neq 0$, sử dụng thống kê:

$$t_j = \frac{\hat{\beta}_j}{\hat{\sigma} \sqrt{[(\mathbf{X}^T \mathbf{X})^{-1}]_{jj}}} \sim t_{n-p-1} \quad \text{dưới } H_0.$$

Sai số chuẩn của $\hat{\beta}_j$ là $\text{se}(\hat{\beta}_j) = \hat{\sigma} \sqrt{[(\mathbf{X}^T \mathbf{X})^{-1}]_{jj}}$. Khoảng tin cậy $(1 - \alpha)$ cho β_j :

$$\hat{\beta}_j \pm t_{\alpha/2, n-p-1} \cdot \text{se}(\hat{\beta}_j).$$

Định lý 1.5: Kiểm định F cho toàn mô hình

[4, 9] Để kiểm định $H_0 : \beta_1 = \dots = \beta_p = 0$ (không biến nào có tác động), sử dụng:

$$F = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n-p-1)} = \frac{R^2/p}{(1-R^2)/(n-p-1)} \sim F_{p, n-p-1} \quad \text{dưới } H_0.$$

Bác bỏ H_0 khi $p\text{-value} = P(F_{p, n-p-1} \geq F_{\text{obs}}) < \alpha$.

Kiểm định t đánh giá từng biến *riêng lẻ* có điều kiện trên các biến khác; kiểm định F đánh giá *toàn bộ* mô hình cùng lúc. Khi các biến có tương quan cao, t -test của từng biến có thể yếu nhưng F -test vẫn có thể mạnh — hai kiểm định bổ sung cho nhau và cần được xem xét cùng nhau.

Table 1: Tóm tắt các chỉ số đánh giá mô hình hồi quy

Chỉ số	Công dụng chính	Ngưỡng kỳ vọng
R^2	Tỷ lệ phương sai được giải thích.	Càng gần 1 càng tốt
$\bar{R}^2$	Lựa chọn số biến tối ưu (phạt overfitting).	Tối đa hóa
t_j -test	Ý nghĩa thống kê từng hệ số β_j .	$ t_j > t_{\alpha/2, n-p-1}$
F -test	Ý nghĩa tổng thể của mô hình.	$p\text{-value} < \alpha$
$\hat{\sigma}$	Độ chính xác tuyệt đối của dự đoán.	Càng nhỏ càng tốt

2 Cài Đặt Python

2.1 Imports, hằng số, dataclass và các hàm phụ trợ

```
1 """
2 OLS Solvers -- Data Fitting and Ordinary Least Squares
3
4 Core functions (pure Python, no NumPy):
5     ols_fit(X, y)          -- OLS coefficients and noise-variance estimate.
6     hat_matrix(X, tol)    -- Projection matrix H and property checks.
7
8 NumPy is used only in the __main__ self-test block for verification.
9 """
10
11 import copy
12 from dataclasses import dataclass
13 from math import sqrt
14 from time import perf_counter
15 from typing import List, Optional, Tuple
16
17
18 EPS_PIVOT          = 1e-12    # pivot threshold for singularity detection
19 DEFAULT_TOL_IDEM  = 1e-10    # Frobenius tolerance for idempotent check
20
21 @dataclass
22 class OLSResult:
```

```
23     """Unified result container for ols_fit."""
24     method: str          # "OLS-NormalEquations"
25     beta_hat: List[float] # shape (p+1,)
26     sigma2_hat: float    # RSS / (n - p - 1)
27     y_hat: List[float]   # shape (n,)
28     residuals: List[float] # shape (n,)
29     rss: float           # ||e||^2
30     dof: int             # n - p - 1
31     success: bool
32     runtime_sec: float
33     message: str
34
35
36 @dataclass
37 class HatMatrixResult:
38     """Unified result container for hat_matrix."""
39     method: str          # "HatMatrix"
40     H: List[List[float]] # shape (n, n)
41     sym_err: float       # ||H - H^T||_inf
42     idem_err: float      # ||H^2 - H||_F
43     rank_H: int          # expected p+1
44     trace_H: float       # expected p+1
45     success: bool
46     runtime_sec: float
47     message: str
48
49
50 def _validate_inputs(
51     X: List[List[float]], y: Optional[List[float]] = None
52 ) -> Tuple[List[List[float]], Optional[List[float]]]:
53     """Validate X and y; return float copies."""
54     if not isinstance(X, list) or not X or not isinstance(X[0], list):
55         raise ValueError("X must be a 2-D list of lists.")
56     n, k = len(X), len(X[0])
57     if any(len(row) != k for row in X):
58         raise ValueError("All rows of X must have the same length.")
```



```
59     if n < k:
60         raise ValueError(f"n={n} must be >= k={k} (underdetermined system).")
61     X_copy = [[float(v) for v in row] for row in X]
62     if y is None:
63         return X_copy, None
64     if len(y) != n:
65         raise ValueError(f"len(y)={len(y)} must equal n={n}.")
66     return X_copy, [float(v) for v in y]
67
68
69 def _transpose(A: List[List[float]]) -> List[List[float]]:
70     m, n = len(A), len(A[0])
71     return [[A[i][j] for i in range(m)] for j in range(n)]
72
73
74 def _matmul(A: List[List[float]], B: List[List[float]]) -> List[List[float]]:
75     m, k, n = len(A), len(A[0]), len(B[0])
76     return [[sum(A[i][p] * B[p][j] for p in range(k)) for j in range(n)]
77             for i in range(m)]
78
79
80 def _matvec(A: List[List[float]], x: List[float]) -> List[float]:
81     return [sum(A[i][j] * x[j] for j in range(len(x))) for i in range(len(A))]
82
83
84 def _vecsub(a: List[float], b: List[float]) -> List[float]:
85     return [a[i] - b[i] for i in range(len(a))]
86
87
88 def _dot(x: List[float], y: List[float]) -> float:
89     return sum(xi * yi for xi, yi in zip(x, y))
90
91
92 def _mat_sub(A: List[List[float]], B: List[List[float]]) -> List[List[float]]:
93     return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]
94
```

```
95
96 def _norm_fro(A: List[List[float]]) -> float:
97     return sqrt(sum(A[i][j] ** 2 for i in range(len(A)) for j in range(len(A[0]))))
98
99
100 def _norm_inf_mat(A: List[List[float]]) -> float:
101     return max(abs(A[i][j]) for i in range(len(A)) for j in range(len(A[0])))
102
103
104 def _trace(A: List[List[float]]) -> float:
105     return sum(A[i][i] for i in range(len(A)))
106
107
108 def _mat_inv(A: List[List[float]]) -> List[List[float]]:
109     """Invert A via Gauss-Jordan on the augmented matrix [A | I]."""
110     n = len(A)
111     aug = [A[i][:] + [1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
112     for col in range(n):
113         pivot_row = max(range(col, n), key=lambda r: abs(aug[r][col]))
114         if abs(aug[pivot_row][col]) < EPS_PIVOT:
115             raise ValueError(f"Singular matrix: zero pivot at column {col}.")
116         aug[col], aug[pivot_row] = aug[pivot_row], aug[col]
117         pivot_val = aug[col][col]
118         aug[col] = [v / pivot_val for v in aug[col]]
119         for row in range(n):
120             if row == col:
121                 continue
122             f = aug[row][col]
123             aug[row] = [aug[row][j] - f * aug[col][j] for j in range(2 * n)]
124     return [row[n:] for row in aug]
125
126
127 def _mat_rank(A: List[List[float]], tol: float = EPS_PIVOT) -> int:
128     """Compute rank of A via Gaussian elimination."""
129     m, n = len(A), len(A[0])
130     mat = copy.deepcopy(A)
```

```
131 rank, used_row = 0, 0
132 for col in range(n):
133     pivot_row = next((r for r in range(used_row, m) if abs(mat[r][col]) > tol),
134                     None)
135     if pivot_row is None:
136         continue
137     mat[used_row], mat[pivot_row] = mat[pivot_row], mat[used_row]
138     for r in range(used_row + 1, m):
139         if abs(mat[used_row][col]) < tol:
140             break
141         f = mat[r][col] / mat[used_row][col]
142         mat[r] = [mat[r][j] - f * mat[used_row][j] for j in range(n)]
143     rank += 1
144     used_row += 1
145 return rank
```

Listing 1: Imports, hằng số, dataclass và các hàm phụ trợ

2.2 Hàm `ols_fit(X, y)`

2.2.1 Mô tả

Hàm `ols_fit` tính toán tường minh theo công thức Normal Equations: (1) tính $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$; (2) tính phần dư và RSS; (3) ước lượng $\hat{\sigma}^2 = \text{RSS} / (n - p - 1)$.

Hàm nghịch đảo `_mat_inv` sử dụng phương pháp Gauss–Jordan có chọn trục (partial pivoting) trên ma trận bổ sung $[\mathbf{A} \mid \mathbf{I}]$.

2.2.2 Code

```
1 # Core solvers
2 def ols_fit(X: List[List[float]], y: List[float]) -> OLSResult:
3     """
4     Compute OLS estimate via Normal Equations.
5     """
```

```
6 Model :  $y = X \beta + \epsilon$ ,  $\epsilon \sim (0, \sigma^2 I_n)$ 
7
8 Algorithm:
9     1.  $G = X^T X$ ,  $q = X^T y$ 
10    2. Check rank(G) == k
11    3.  $\beta_{\text{hat}} = G^{-1} q$ 
12    4.  $y_{\text{hat}} = X \beta_{\text{hat}}$ ,  $e = y - y_{\text{hat}}$ ,  $RSS = e.e$ 
13    5.  $\sigma^2_{\text{hat}} = RSS / (n - p - 1)$ 
14
15 Parameters:
16      $X$  : List[List[float]], shape (n, p+1) -- design matrix (first col = 1).
17      $y$  : List[float], shape (n,) -- response vector.
18
19 Returns:
20     OLSResult
21     """
22     start = perf_counter()
23     X_mat, y_vec = _validate_inputs(X, y)
24     n, k = len(X_mat), len(X_mat[0])
25     p = k - 1
26
27     try:
28         # Step 1:  $G = X^T X$ ,  $q = X^T y$ 
29         Xt = _transpose(X_mat)
30         G = _matmul(Xt, X_mat)
31         q = _matvec(Xt, y_vec)
32
33         # Step 2: Check invertibility
34         if _mat_rank(G) < k:
35             return OLSResult(
36                 method="OLS-NormalEquations", beta_hat=[], sigma2_hat=float("nan"),
37                 y_hat=[], residuals=[], rss=float("nan"), dof=n - p - 1,
38                 success=False, runtime_sec=perf_counter() - start,
39                 message="X^T X is not invertible (multicollinearity).",
40             )
41
```

```
42     # Step 3: beta_hat = (X^T X)^{-1} X^T y
43     beta_hat = _matvec(_mat_inv(G), q)
44
45     # Step 4: Residuals and RSS
46     y_hat     = _matvec(X_mat, beta_hat)
47     residuals = _vecsub(y_vec, y_hat)
48     rss       = _dot(residuals, residuals)
49
50     # Step 5: sigma^2 = RSS / (n - p - 1)
51     dof       = n - p - 1
52     sigma2_hat = rss / dof
53
54     return OLSResult(
55         method="OLS-NormalEquations", beta_hat=beta_hat, sigma2_hat=sigma2_hat,
56         y_hat=y_hat, residuals=residuals, rss=rss, dof=dof, success=True,
57         runtime_sec=perf_counter() - start,
58         message=f"Solved. RSS={rss:.6g}, sigma2={sigma2_hat:.6g}, dof={dof}.",
59     )
60
61 except Exception as exc:
62     return OLSResult(
63         method="OLS-NormalEquations", beta_hat=[], sigma2_hat=float("nan"),
64         y_hat=[], residuals=[], rss=float("nan"), dof=n - p - 1,
65         success=False, runtime_sec=perf_counter() - start,
66         message=f"ols_fit failed: {exc}",
67     )
```

Listing 2: Hàm ols_fit

2.3 Hàm hat_matrix(X)

2.3.1 Mô tả

Hàm hat_matrix tính $\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ và tự động kiểm tra tính idempotent bằng cách đo $\|\mathbf{H}^2 - \mathbf{H}\|_F$.

2.3.2 Code

```
1 def hat_matrix(X: List[List[float]], tol: float = DEFAULT_TOL_IDEM) -> HatMatrixResult:
2     """
3     Compute the hat matrix  $H = X (X^T X)^{-1} X^T$  and verify properties.
4
5     H is the orthogonal projector onto  $C(X)$ :
6          $y_{\text{hat}} = H y, \quad e = (I - H) y$ 
7
8     Properties checked:
9         (i) Idempotent :  $\|H^2 - H\|_F \leq \text{tol}$ 
10        (ii) Symmetric :  $\|H - H^T\|_{\text{inf}}$  (reported)
11        (iii) rank(H) = p+1, tr(H) = p+1
12
13    Algorithm:
14        1.  $G_{\text{inv}} = (X^T X)^{-1}$ 
15        2.  $H = X G_{\text{inv}} X^T$ 
16        3. sym_err, idem_err, rank_H, trace_H
17
18    Parameters:
19        X : List[List[float]], shape (n, p+1)
20        tol : float -- idempotent tolerance (default 1e-10)
21
22    Returns:
23        HatMatrixResult
24    """
25    start = perf_counter()
26    X_mat, _ = _validate_inputs(X)
27    n, k = len(X_mat), len(X_mat[0])
28
29    try:
30        # Step 1:  $(X^T X)^{-1}$ 
31        Xt = _transpose(X_mat)
32        G_inv = _mat_inv(_matmul(Xt, X_mat))
33
34        # Step 2:  $H = X (X^T X)^{-1} X^T$ 
```

```
35     H = _matmul(_matmul(X_mat, G_inv), Xt)
36
37     # Step 3: Property checks
38     sym_err = _norm_inf_mat(_mat_sub(H, _transpose(H)))
39     idem_err = _norm_fro(_mat_sub(_matmul(H, H), H))
40     rank_H = _mat_rank(H)
41     trace_H = _trace(H)
42
43     ok = idem_err <= tol
44     return HatMatrixResult(
45         method="HatMatrix", H=H, sym_err=sym_err, idem_err=idem_err,
46         rank_H=rank_H, trace_H=trace_H, success=ok,
47         runtime_sec=perf_counter() - start,
48         message=(
49             f"{'[OK]' if ok else '[WARN]'} ||H2-H||_F={idem_err:.2e}, "
50             f"||H-HT||_inf={sym_err:.2e}, "
51             f"rank={rank_H} (exp {k}), tr={trace_H:.4f} (exp {k})."
52         ),
53     )
54
55 except Exception as exc:
56     return HatMatrixResult(
57         method="HatMatrix", H=[[]], sym_err=float("nan"), idem_err=float("nan"),
58         rank_H=-1, trace_H=float("nan"), success=False,
59         runtime_sec=perf_counter() - start,
60         message=f"hat_matrix failed: {exc}",
61     )
62
63
64 def run_ols_analysis(
65     X: List[List[float]], y: List[float], tol_idem: float = DEFAULT_TOL_IDEM
66 ) -> dict:
67     """Run ols_fit + hat_matrix and return {'ols': OLSResult, 'hat':
68     HatMatrixResult}."""
69     return {"ols": ols_fit(X, y), "hat": hat_matrix(X, tol=tol_idem)}
```

Listing 3: Ham hat_matrix

2.4 Kiểm thử và kiểm chứng

```
1 # Self-test
2 if __name__ == "__main__":
3     import sys, dataclasses
4     import numpy as np
5
6     if hasattr(sys.stdout, "reconfigure"):
7         sys.stdout.reconfigure(encoding="utf-8")
8
9     np.random.seed(42)
10    n_obs, p_feat = 20, 2
11    beta_true = np.array([2.0, 1.0, -0.5])
12    X_raw = np.random.randn(n_obs, p_feat)
13    X_np = np.column_stack([np.ones(n_obs), X_raw])
14    y_np = X_np @ beta_true + 0.5 * np.random.randn(n_obs)
15
16    X_list, y_list = X_np.tolist(), y_np.tolist()
17    results = run_ols_analysis(X_list, y_list)
18
19    for name, res in results.items():
20        print(f"\n{'='*55}\n {name.upper()}\n{'='*55}")
21        for field, val in dataclasses.asdict(res).items():
22            if isinstance(val, list):
23                if val and isinstance(val[0], list):
24                    print(f" {field}: <{len(val)}x{len(val[0])} matrix>")
25                elif len(val) <= 6:
26                    print(f" {field}: {[round(v, 6) for v in val]}")
27                else:
28                    print(f" {field}: {[round(v, 6) for v in val[:4]]} ...
29                        (n={len(val)})")
29            else:
```



```
30         print(f" {field}: {val}")
31
32     # --- Verification against NumPy ---
33     ols_res = results["ols"]
34     hat_res = results["hat"]
35     print(f"\n{'='*55}\n  Verification\n{'='*55}")
36
37     beta_np, _, _, _ = np.linalg.lstsq(X_np, y_np, rcond=None)
38     diff_beta = float(np.max(np.abs(np.array(ols_res.beta_hat) - beta_np)))
39     print(f"  ols_fit    ||beta_hat - numpy||_inf = {diff_beta:.2e}  {'PASSED' if
40         diff_beta < 1e-8 else 'FAILED'}")
41
42     H_np = X_np @ np.linalg.inv(X_np.T @ X_np) @ X_np.T
43     diff_H = float(np.max(np.abs(np.array(hat_res.H) - H_np)))
44     print(f"  hat_matrix ||H_ours - H_numpy||_inf = {diff_H:.2e}  {'PASSED' if diff_H <
45         1e-8 else 'FAILED'}")
46
47     print(f"\n  beta_true : {beta_true.tolist()}")
48     print(f"  beta_hat   : {[round(v, 6) for v in ols_res.beta_hat]}")
```

Listing 4: Self-test va verification

References

- [1] S. M. Stigler, “Statistics and the question of standards,” *Journal of Research of the National Institute of Standards and Technology*, vol. 101, no. 6, p. 779, 1996. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/jres/101/6/j6stig.pdf>
- [2] R. L. Plackett, “Studies in the history of probability and statistics. xxix: The discovery of the method of least squares,” *Biometrika*, vol. 59, no. 2, p. 239, 1972.
- [3] G. Strang, *Introduction to Linear Algebra*, 6th ed. Wellesley-Cambridge Press, 2023.
- [4] J. M. Wooldridge, *Introductory Econometrics: A Modern Approach*, 5th ed. South-Western Cengage Learning, 2013.
- [5] W. H. Greene, *Econometric Analysis*, 7th ed. Prentice Hall, 2012.
- [6] G. H. Golub and C. F. V. Loan, *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013.
- [7] L. N. Trefethen and D. B. III, *Numerical Linear Algebra*. SIAM, 1997.
- [8] D. C. Hoaglin and R. E. Welsch, “The hat matrix in regression and anova,” 1978.
- [9] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning*, 1st ed. Springer, 2013.